



Harish Kumar S 251801090-2025-AIDS ▾

**H2****Started on** Tuesday, 25 August 2026, 4:44 PM**State** Finished**Completed on** Tuesday, 25 August 2026, 4:45 PM**Time taken** 1 min 4 secs**Marks** 1.00/1.00**Grade** 10.00 out of 10.00 (100%)

**Question 1** | Correct | Mark 1.00 out of 1.00

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child  $i$  has a greed factor  $g[i]$ , which is the minimum size of a cookie that the child will be content with; and each cookie  $j$  has a size  $s[j]$ . If  $s[j] \geq g[i]$ , we can assign the cookie  $j$  to the child  $i$ , and the child  $i$  will be content. Your goal is to maximize the number of your content children and output the maximum number.

**Example 1:****Input:**

```
3
1 2 3
2
1 1
```

**Output:**

```
1
```

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.

And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.

You need to output 1.

**Constraints:**

$1 \leq g.length \leq 3 \times 10^4$

$0 \leq s.length \leq 3 \times 10^4$

$1 \leq g[i], s[j] \leq 2^{31} - 1$

**Answer:** (penalty regime: 0 %)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Comparison function for qsort
5 int compare(const void *a, const void *b) {
6     long long x = *(const long long *)a;
7     long long y = *(const long long *)b;
8     if (x < y) return -1;
9     if (x > y) return 1;
10    return 0;
11 }
12
13 int main() {
14     int n, m;
15
16     // Read children count and greed factors
17     if (scanf("%d", &n) != 1) return 0;
18     long long g[n];
19     for (int i = 0; i < n; i++) {
20         scanf("%lld", &g[i]);
21     }
22
23     // Read cookies count and sizes
24     if (scanf("%d", &m) != 1) return 0;
25     long long s[m];
26     for (int i = 0; i < m; i++) {
27         scanf("%lld", &s[i]);
28     }
29
30     // Sort both arrays
31     qsort(g, n, sizeof(long long), compare);
32     qsort(s, m, sizeof(long long), compare);
33
34     // Two pointer greedy approach
35     int child_ptr = 0;
36     int cookie_ptr = 0;
37
38     while (child_ptr < n && cookie_ptr < m) {
```

```
39     if (s[cookie_ptr] >= g[child_ptr]) {
40         child_ptr++; // Satisfy child and move to next child
41     }
42     cookie_ptr++; // Move to next cookie regardless
43 }
44
45 printf("%d\n", child_ptr);
46 return 0;
47 }
```

|   | Input | Expected | Got |   |
|---|-------|----------|-----|---|
| ✓ | 2     | 2        | 2   | ✓ |
|   | 1 2   |          |     |   |
|   | 3     |          |     |   |
|   | 1 2 3 |          |     |   |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)